PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Architecture and Design Specification

Smart Complaint Management System

Version 1.1

Prepared: 24-10-2026

Team Members: Dhawal Pathak — PES1UG24CS151

GANESH M — PES1UG24CS168

Harshitha P — PES1UG24CS186

Revision History

Version	Date	Author	Reason for changes
1.1	24-10-2026	Project Team	Aligned with the implementation on `main` (commit `2cc96e3`) for Issue #69. Replaced the planned `/api/...` design and the undecided technology stack with what the code implements, and recorded verified limitations.

1. Introduction

1.1 Purpose

Defines the architecture and detailed design of the Smart Complaint Management System and translates SRS requirements into modules, interfaces, data flow, security controls and testable design elements.

1.2 Scope

Authentication, complaint creation/tracking, administrator management, persistence, authorization and server-rendered page communication.

1.3 Baseline, pending work and evidence

This version describes the code on the `main` branch at commit `2cc96e3` (local reference). Open pull requests are pending work and are not described as implemented: #83 (complaint detail page with owner check), #84 (shared styling for user and error pages), #85 (configurable production database support), #86 (registration validation) and #87 (home page and user dashboard).

Every statement about the implementation is labelled by how it was established:

- **Code inspection** means it was read from the source files on `main`.
- **Git history** means it was read from the commit history.
- **Runtime** means it was observed by running the application or its tests. No runtime verification was performed for this version of the document: no tests were run and no routes were executed.

No statement is made about any hosted or deployed environment.

2. Architecture

2.1 Goals and Constraints

- Clear separation of presentation, API/application, business and data layers.
- Secure authentication and role-based authorization.
- Simple architecture suitable for a student mini-project.
- Extensible enough to add users/categories without redesign.

2.2 Architecture Pattern and Rationale

Version 1.0 proposed a layered architecture (presentation, API/application, business/service, data access, database). The implementation is a single Flask application that renders HTML pages on the server. Route functions in three blueprints contain both the application and the business logic; there is no separate service layer. Data access uses Flask-SQLAlchemy models directly from the route functions, and the database is SQLite. This keeps the project simple and avoids the operational complexity of microservices. (Code inspection.)

2.3 Component Diagram — Logical Components

The logical components of version 1.0 are kept, with the place each one is implemented on `main`.

Component	Responsibility	Implementation on `main` (code inspection)
Web Client/UI	User/admin screens	Jinja2 templates in `templates/` and one stylesheet, `static/style.css`
Authentication Module	Registration/login/session	`auth.py`: `register`, `login`, `logout`; Flask session cookie
Complaint Service	Create/validate/retrieve complaints	`complaints.py`: `complaint`, `complaints`

Component	Responsibility	Implementation on `main` (code inspection)
Admin Service	Privileged listing/filtering/status/resolution	`admin.py`: `admin_dashboard`, `admin_complaint`
Authorization Middleware	Authentication and role checks	No separate middleware. `admin.admin_only()` is called at the start of each admin route; user routes check `session` directly
Data Access Layer	Database operations	Flask-SQLAlchemy models in `models.py` and the `db` object in `extensions.py`
Database	Persistent user/complaint data	SQLite file, `sqlite:///complaints.db` in `app.py`

The Flask application is created in `app.py`, which registers the `auth`, `complaint_bp` and `admin` blueprints, the CSRF check, the 403/404/500 error handlers, and creates the tables with `db.create_all()` when the module is imported.

2.4 Component Descriptions

- Web client presents role-specific pages rendered by the server.
- The authentication module validates credentials and stores the user id, name and role in the session.
- The complaint routes create complaints for the logged-in user and list that user's complaints.
- The admin routes list, filter and update all complaints and are restricted to the admin role.
- Authorization is enforced inside the routes, not by a separate layer.
- Models and the shared `db` object isolate database access.
- The database stores persistent records.

2.5 Traceability to Requirements

SRS requirement groups and the modules that implement them. Requirement-to-test traceability is in Appendix B of SRS v1.1.

SRS Requirements	Implementing modules (code inspection)	Notes
FR-01, FR-02	`auth.py`	Registration checks only that fields are non-empty
FR-03, FR-04, FR-05, FR-06	`complaints.py`, `models.py`, user templates	No complaint detail page on `main` (FR-06 shows status in the list only)
FR-07	`admin.py` (`admin_only`)	
FR-08, FR-09, FR-10	`admin.py` (`admin_dashboard`, `admin_complaint`)	No enforced transition graph; see section 3.8
FR-11	`models.py`, `extensions.py`, SQLite setting in `app.py`	Local SQLite file only
FR-12	Error messages in `auth.py`, `complaints.py`, `admin.py`; error templates	
NFR-03, SEC-03	Werkzeug password hashing in `auth.py`	
NFR-04, SEC-01, SEC-02	`admin.admin_only()`, session checks in `complaints.py`	
NFR-08	Blueprint split of `auth.py`, `complaints.py`, `admin.py`	

2.6 Security Architecture

Concern	Control implemented (code inspection)	Not implemented / limitation
Spoofing	Passwords verified with `check_password_hash`; login failures use one generic message	No limit on repeated failed logins (Issue #70 is open)
Session handling	Flask signed-cookie session; `HttpOnly`; `SameSite=Lax`; `Secure` only when `SESSION_COOKIE_SECURE=1`; session cleared at logout	A 30-minute `PERMANENT_SESSION_LIFETIME` is configured, but no code marks the session permanent, so idle expiry is not applied; the session is not reset at login
Cross-site request forgery	Token stored in the session, hidden field in every form, every POST without a valid token is rejected with 400	Logout is a GET request and is not covered by the check
Broken access control	Admin routes return 403 for non-admins; complaint routes redirect logged-out users to the login page	None identified
Injection	Database access through SQLAlchemy with bound parameters; Jinja2 autoescaping of template output	Complaint fields are checked for non-empty only, with no length rules
Credential disclosure	Passwords stored as Werkzeug hashes; `SECRET_KEY` read from the environment and the application refuses to start without it	Registration reports "Email already registered", which reveals that an address has an account
Information exposure	Custom 403, 404 and 500 pages without stack traces; debug mode only when `FLASK_DEBUG=1`	No application logging is configured
Transport security	`SESSION_COOKIE_SECURE` switch exists	HTTPS is not part of this repository and is not described here

2.7 Technology Stack

Version 1.0 left the stack undecided. The stack in the repository is:

Layer	Technology	Source
Language	Python (version not declared in the repository)	`*.py`
Web framework	Flask 3.1.3	`requirements.txt`
ORM	Flask-SQLAlchemy 3.1.1 on SQLAlchemy 2.1.3	`requirements.txt`, `extensions.py`
Database	SQLite	`app.py` (`sqlite:///complaints.db`)
Templates	Jinja2 3.1.6, server-rendered HTML	`requirements.txt`, `templates/`
Password hashing	Werkzeug 3.1.9 (`generate_password_hash`, `check_password_hash`)	`requirements.txt`, `auth.py`
Styling	One stylesheet; no client-side framework in the templates	`static/style.css`
WSGI server	`gunicorn` 26.2.0 is listed in `requirements.txt`; no start command or process configuration is in the repository	`requirements.txt`
Tests	pytest 9.1.1	`requirements.txt`, `tests/test_app.py`

2.8 Local Configuration

Setting	Required	Effect (code inspection of `app.py`)
`SECRET_KEY` environment variable	Yes	Signs the session cookie. The application raises `RuntimeError` at start-up if it is not set
`FLASK_DEBUG`	No	`python app.py` runs in debug mode only when the value is `1`
`SESSION_COOKIE_SECURE`	No	Sets the `Secure` flag on the session cookie when the value is `1`

The database is `sqlite:///complaints.db`. Flask-SQLAlchemy places a relative SQLite file in the application's instance folder (framework behaviour), and `.gitignore` excludes `instance/`. Tables are created at start-up with `db.create_all()`; there are no migrations. An administrator account is created by registering a normal account and then running `python make_admin.py <email>`, which promotes that user.

3. Design

3.1 Design Overview

A browser requests a page. The Flask route runs the CSRF check for POST requests, checks the session where the route requires login or the admin role, reads or writes the database through the models, and returns a rendered template, a redirect or a short text response. All of this was established by code inspection.

3.2 Sequence Diagram 1 — Submit Complaint

Step	Component	Interaction (code inspection)
1	User	Opens `/complaint`
2	`complaints.complaint`	If `user_id` is not in the session, redirects to `/login`; otherwise renders `complaint.html` with a CSRF token
3	User	Enters title, category and description and submits (POST)
4	`app.check_csrf`	Rejects the request with 400 if the token is missing or wrong
5	`complaints.complaint`	Trims the three fields; if any is empty, shows "Title, description, and category are required."
6	`complaints.complaint`	Creates a `Complaint` with the session user id (status defaults to `Pending`) and commits
7	`complaints.complaint`	Returns the text "Complaint submitted successfully!"

3.3 Sequence Diagram 2 — Admin Resolves Complaint

Step	Component	Interaction (code inspection)
1	Admin	Logs in; `auth.login` redirects an admin to `/admin`
2	`admin.admin_dashboard`	`admin_only()` returns 403 unless the session role is `admin`; lists complaints newest first with per-status counts and optional filters
3	Admin	Follows the Manage link to `/admin/complaint/<id>` (see limitation 8 in section 3.8: this page cannot render)
4	Admin	Chooses a status and optional remarks and submits (POST); the CSRF check runs first
5	`admin.admin_complaint`	Rejects a status that is not in the status list ("Invalid status.") and a Resolved status without remarks ("Resolution remarks are required.")
6	`admin.admin_complaint`	Sets the status, stores the remarks if any were entered, and commits; `updated_at` is refreshed by the model
7	`admin.admin_complaint`	Redirects to `/admin`

3.4 Routes

The application is server-rendered. It has no JSON API and no `/api/...` routes; the endpoints of version 1.0 do not exist. Routes on `main` (code inspection):

Path	Methods	Endpoint	Access	Behaviour
`/`	GET	`home`	Public	Returns the plain text "Smart Complaint Management System"
`/register`	GET, POST	`auth.register`	Public	POST requires name, email and password to be non-empty; an email that already exists (exact match) is rejected; success returns text
`/login`	GET, POST	`auth.login`	Public	Admin success redirects to `/admin`; user success returns "Welcome, name!"; failure shows "Invalid email or password."
`/logout`	GET	`auth.logout`	Anyone	Clears the session and returns text
`/complaint`	GET, POST	`complaints.complaint`	Logged in; otherwise redirect to `/login`	Form and submission as in section 3.2
`/complaints`	GET	`complaints.complaints`	Logged in; otherwise redirect to `/login`	Lists the session user's complaints with status, timestamps and resolution
`/admin`	GET	`admin.admin_dashboard`	Admin only; otherwise 403, including for logged-out users	Counts per status, complaint list, optional `status` and `category` query filters
`/admin/complaint/<int:complaint_id>`	GET, POST	`admin.admin_complaint`	Admin only; 404 if the complaint does not exist	Shows a complaint and accepts a status and remarks (see limitation 8)
`/static/<path:filename>`	GET	`static`	Public	Serves `static/style.css`

Every POST request is checked by `app.check_csrf` before the route runs, and is answered with 400 if the token is missing or does not match.

3.4a Data Model

Two models are defined in `models.py` (code inspection). There is no other model.

User

Field	Type	Constraints
`id`	Integer	Primary key
`name`	String(100)	Required
`email`	String(120)	Required, unique
`password`	String(200)	Required; stores the hash
`role`	String(20)	Default `user`; `admin` for administrators

Complaint

Field	Type	Constraints
`id`	Integer	Primary key
`title`	String(200)	Required
`description`	Text	Required
`category`	String(50)	Required, default `Other`; free text on submission
`status`	String(50)	Default `Pending`
`resolution`	Text	Optional
`user_id`	Integer	Foreign key to `user.id`, required
`created_at`	DateTime	Default: current UTC time
`updated_at`	DateTime	Default: current UTC time; refreshed on update

3.4b Status Behaviour

The status values are `Pending`, `In Progress` and `Resolved` (list `STATUSES` in `admin.py`). A new complaint is `Pending`. On the admin complaint form (code inspection):

- The submitted status must be one of the three values; otherwise "Invalid status." is shown.
- A status of `Resolved` requires non-empty resolution remarks; otherwise "Resolution remarks are required." is shown.
- Remarks, when entered, are saved with whatever status is chosen.
- Any of the three statuses can be set from any other, including `Resolved` back to `Pending`. There is no transition graph.
- No record is kept of who changed a complaint or what it was before; only the current `status`, `resolution` and `updated_at` are stored.

3.5 Error Handling, Logging and Monitoring

Condition	Behaviour (code inspection)
Missing or invalid CSRF token	`abort(400)`; the framework's default 400 page is shown
Not an administrator	403 with `templates/403.html`
Unknown URL or complaint id	404 with `templates/404.html`
Unexpected server error	500 with `templates/500.html`; the database session is rolled back
Invalid form input	The form is shown again with a message from the route
401 and 409	Not used
Logging and monitoring	No application logging is configured

3.6 UX Design

- Keep complaint submission concise and clearly labelled.
- Show current status prominently.
- Use consistent status labels and success/error feedback.
- Separate user/admin navigation.

Note for version 1.1: the admin dashboard has its own header with a logout link. A shared navigation bar and home page for users are in pending PR #87 and are not part of `main`. Success feedback after registration, login and complaint submission is plain text.

3.7 Open Issues and Next Steps

Pending pull requests (not merged into `main`): #83 complaint detail page; #84 shared styling for user and error pages; #85 configurable production database support; #86 registration validation; #87 home page and user dashboard.

Open issues taken from the public issue list that affect this design:

- #41 admin complaint page crashes after the module split; #42 fixed category list so the admin category filter works.
- #43 enforce allowed status transitions; #45 record which admin updated a complaint and when.
- #44 automated tests for admin features; #53 integration test report; #54 run pytest in CI.
- #47 logout as a POST request with a CSRF token; #70 limit repeated failed logins.
- #51 user complaint detail page; #52 success messages; #61 logging without secrets.
- #62 database migrations; #73 keep production data across redeploys.

3.8 Verified Limitations

These apply to `main` at commit `2cc96e3`. Items 1, 3 to 8 and 10 were established by code inspection and item 2 by Git history; none was confirmed by running the application for this document. Item 9 depends on how Flask-SQLAlchemy binds its engine, which was also observed when the project's tests were examined earlier; it was not re-run for this document.

1. **No JSON API.** All pages are server-rendered; the `/api/...` endpoints of version 1.0 do not exist.
2. **No `ComplaintUpdate` model.** Only `User` and `Complaint` exist, and no history entries are created. A history table was added by pull request #50 and removed again by pull request #82 (Git history).
3. **No enforced status transition graph.** A status only has to be one of the three allowed values (section 3.4b).
4. **Registration validation is minimal.** Name, email and password only have to be non-empty. Email matching is case-sensitive. Validation is in pending PR #86.
5. **Session idle expiry is not applied.** A lifetime is configured, but the session is never marked permanent, and the session is not reset at login.
6. **Logout is a GET request** and is not covered by the CSRF check.
7. **The admin category filter and the free-text category.** Complaints accept any category text, while the admin filter accepts only a fixed list (`CATEGORIES` in `admin.py`) (Issue #42 is open).
8. **Admin complaint page endpoint defect.** `templates/admin_complaint.html` builds its back link with `url_for('admin_dashboard')`, but the registered endpoint is `admin.admin_dashboard` (section 3.4). Rendering that template therefore fails, and the page cannot be shown. This is a code-inspection finding and was not executed (Issue #41 is open).
9. **Tests and the database.** The fixture in `tests/test_app.py` sets `SQLALCHEMY_DATABASE_URI` after `app.py` has been imported, when the engine already exists, so it does not redirect the tests to a temporary database. The suite should only be run in an isolated copy of the repository.
10. **No durability or hosting claim.** The database is a local SQLite file. This document makes no statement about production persistence, hosting or deployment; none is verified by the repository.